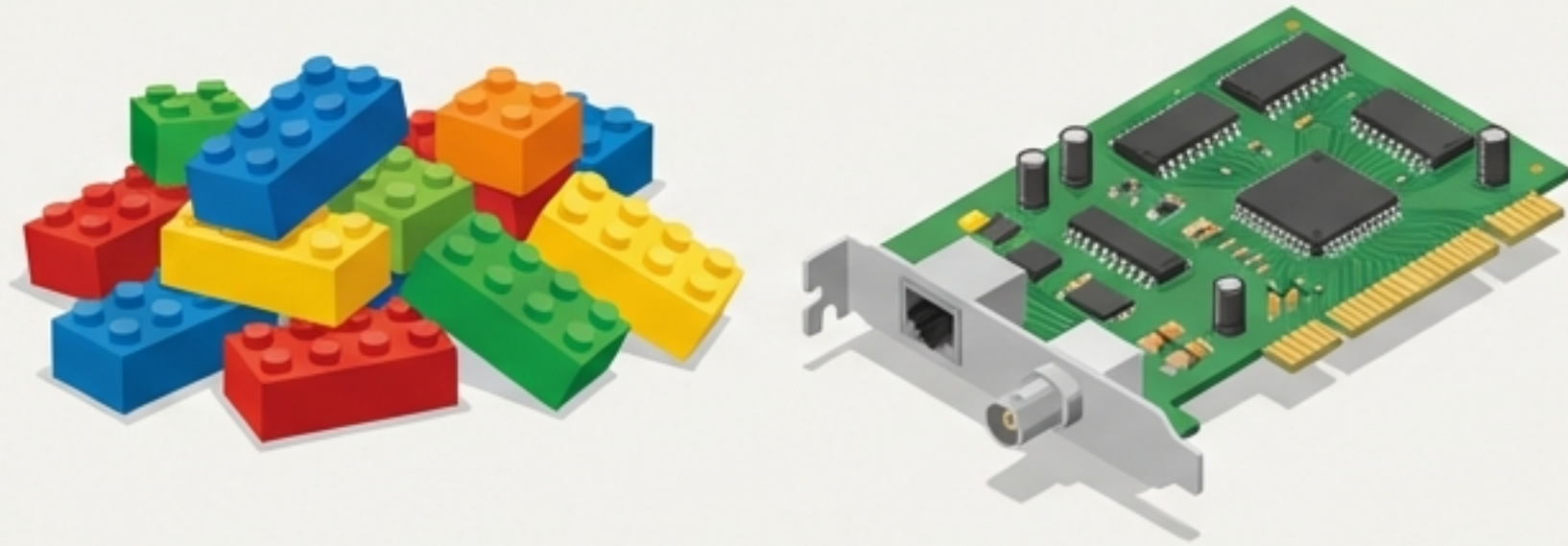




Software Architecture: Modularity & Components

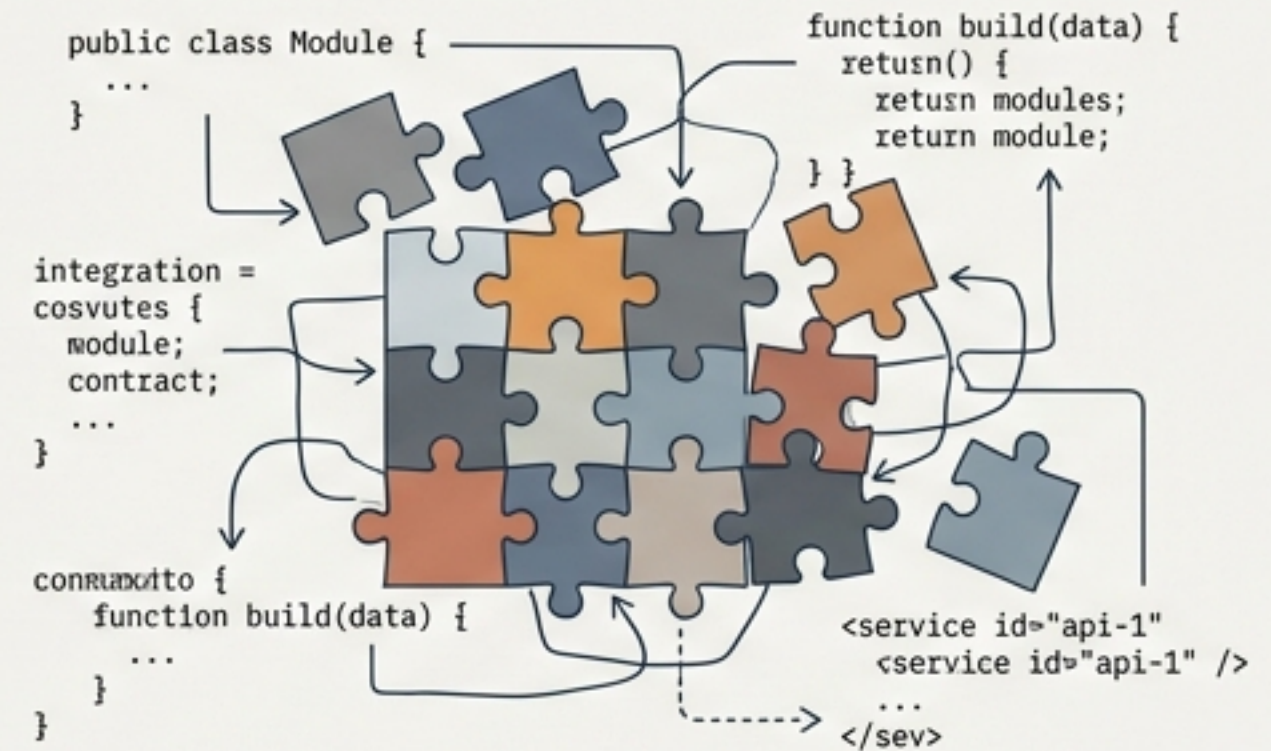
A structural framework for designing, sourcing, and deploying scalable software systems.

The Hardware & Lego Vision



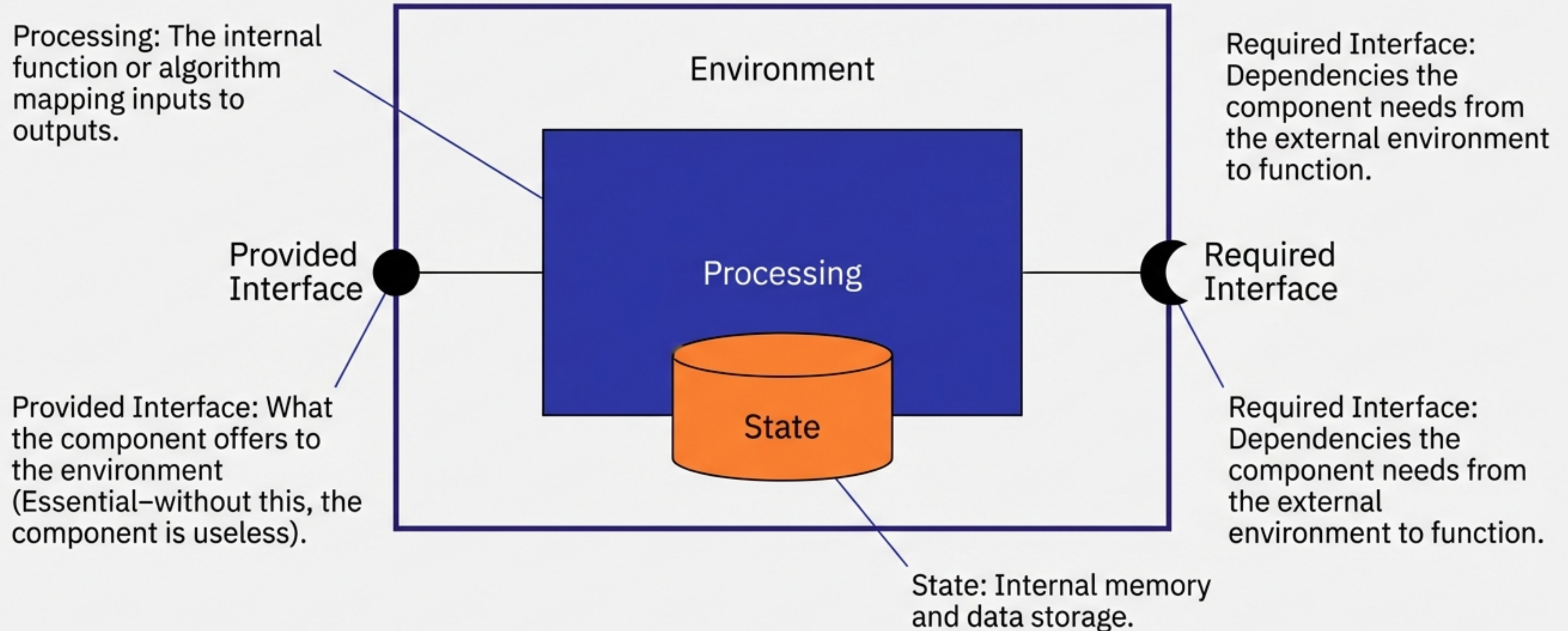
- Reusable units of composition.
- Standardized interfaces (PCI slots, RJ45 jacks, studs).
- Easily “click” together to form larger systems.

The Software Reality



- Highly **heterogeneous** technologies.
- **No** universal “click-together” standard.
- Assembly requires deliberate architectural effort, mapping, and cost.

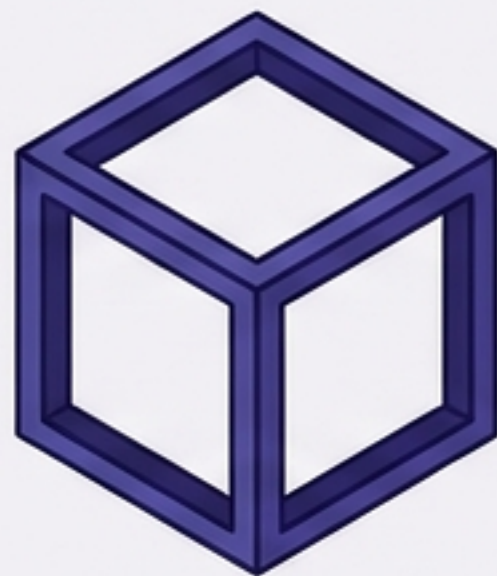
Anatomy of a Software Component



The Four Pillars of Component Theory

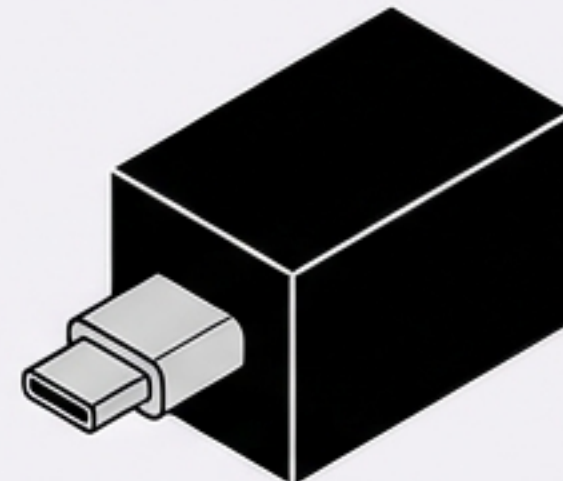
1. Encapsulation

Drawing the boundary. Clearly separating the internal contents from the rest of the system.



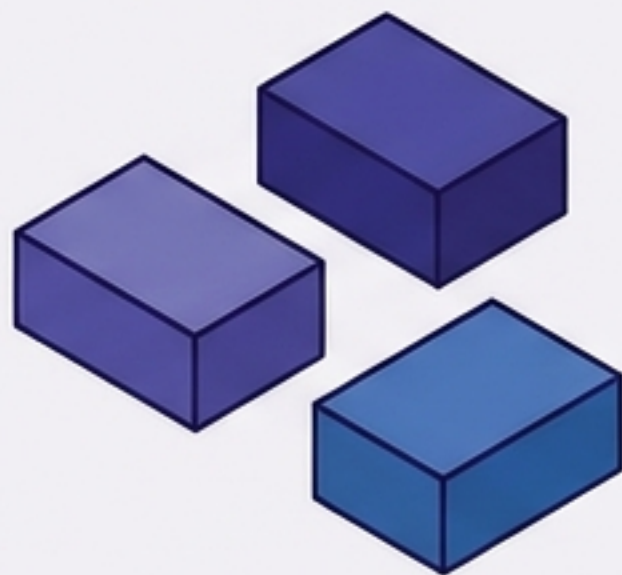
2. Abstraction

Hiding internal implementation details behind the interface contract. You only need to know how to use it, not how it works.



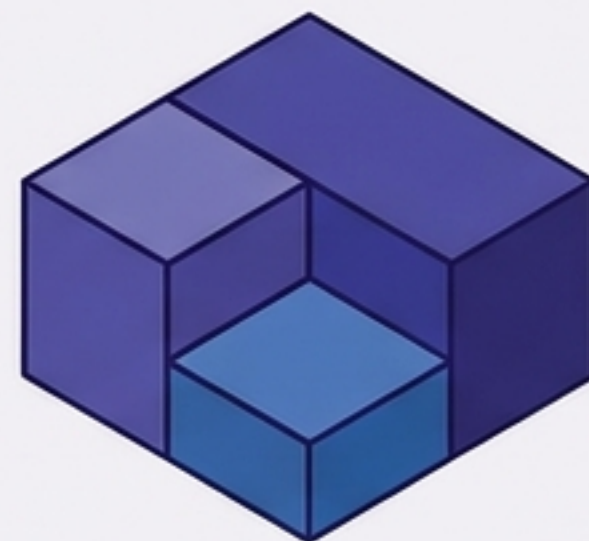
3. Modularity

Acting as an independent, reusable unit of assembly and deployment.



4. Composability

The ability to interconnect with other modules to build systems greater than the sum of their parts.



Component Taxonomy: Application vs. Infrastructure

Application-Specific



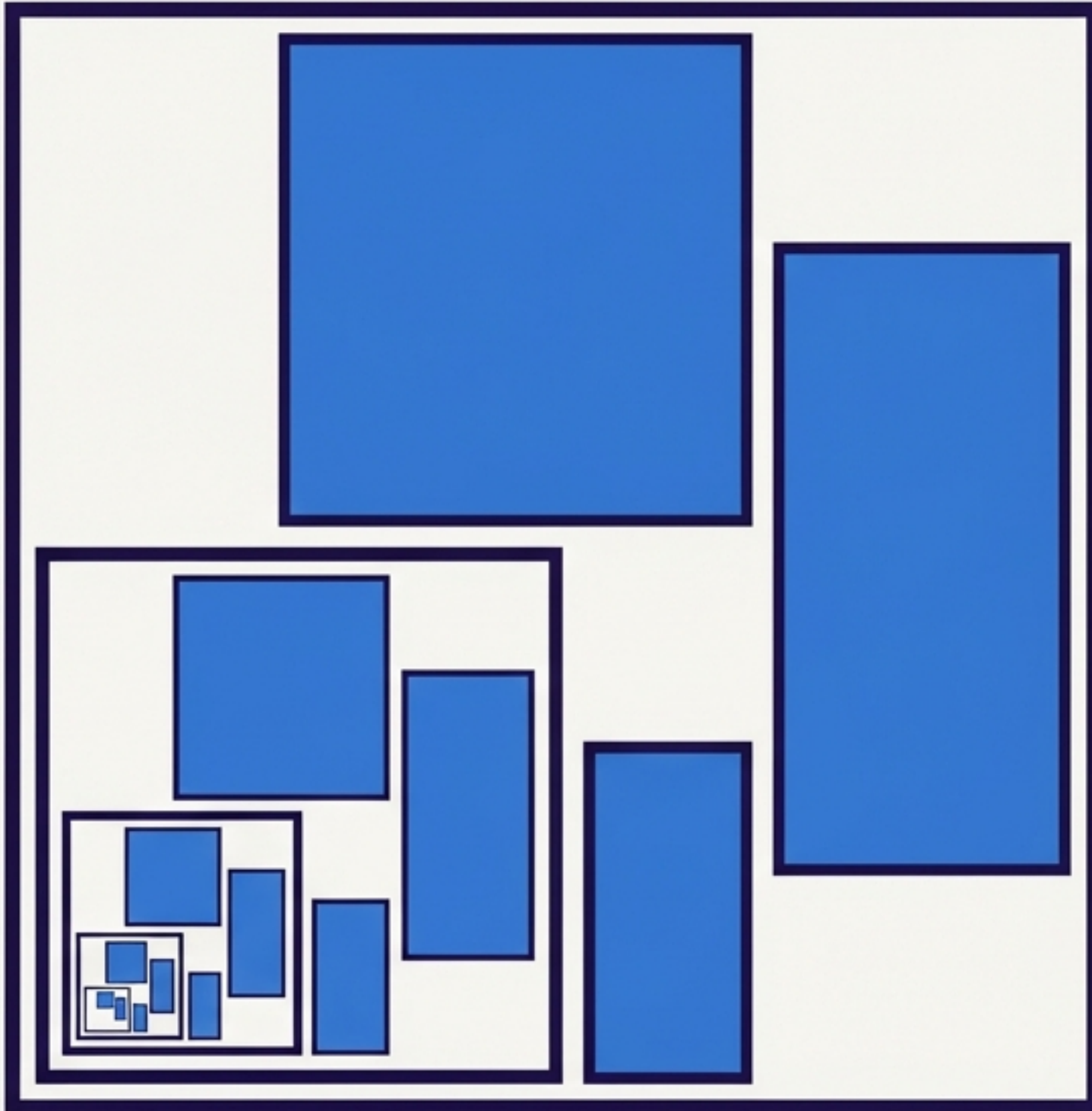
- Directly implements business functionality.
- Requires deep domain knowledge to build.
- Difficult to reuse outside of its specific application domain.

Infrastructure



- Domain-independent and highly reusable.
- Supports extra-functional requirements (scalability, durability).
- Empty by default—requires custom configuration or schema to be useful.

The Fractal Architecture of Components



Recursive Nature:

Components are meant to be composed into larger ones, but inside the black box, they are made of smaller subcomponents.

The Architect's Decision:

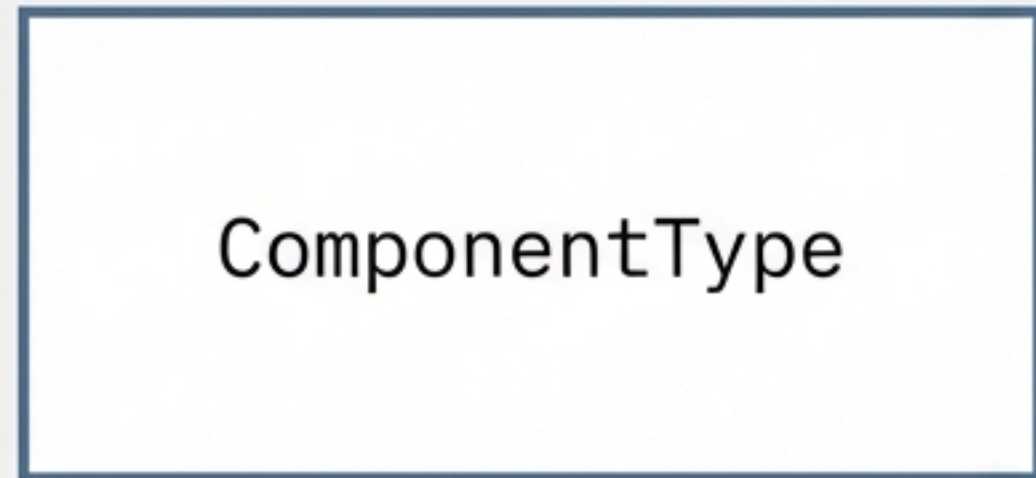
When do we stop splitting?

Rule of Thumb:

Stop the recursion when the boxes become so small that you are no longer making architectural decisions, but simply doing visual programming.

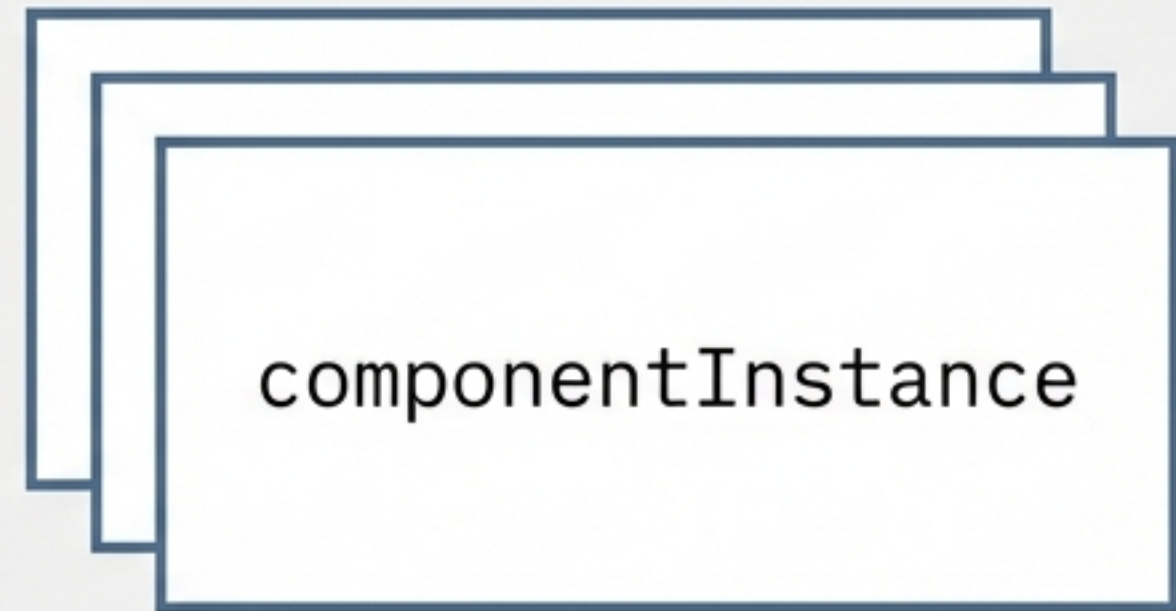
Transitioning from Design to Execution

Design-Time (Logical View)



- Defines the properties shared by a class of components.
- Naming Convention: CamelCase.

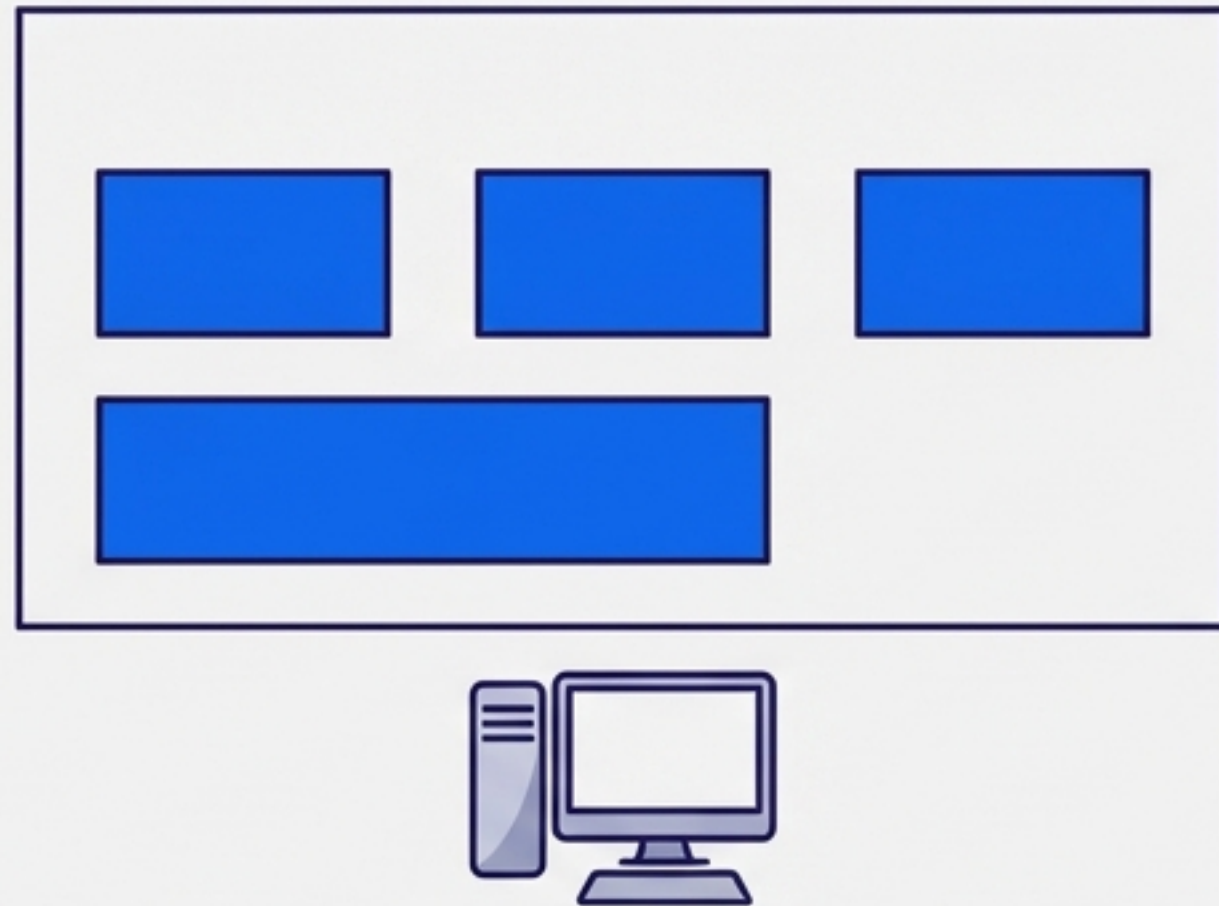
Run-Time (Deployment View)



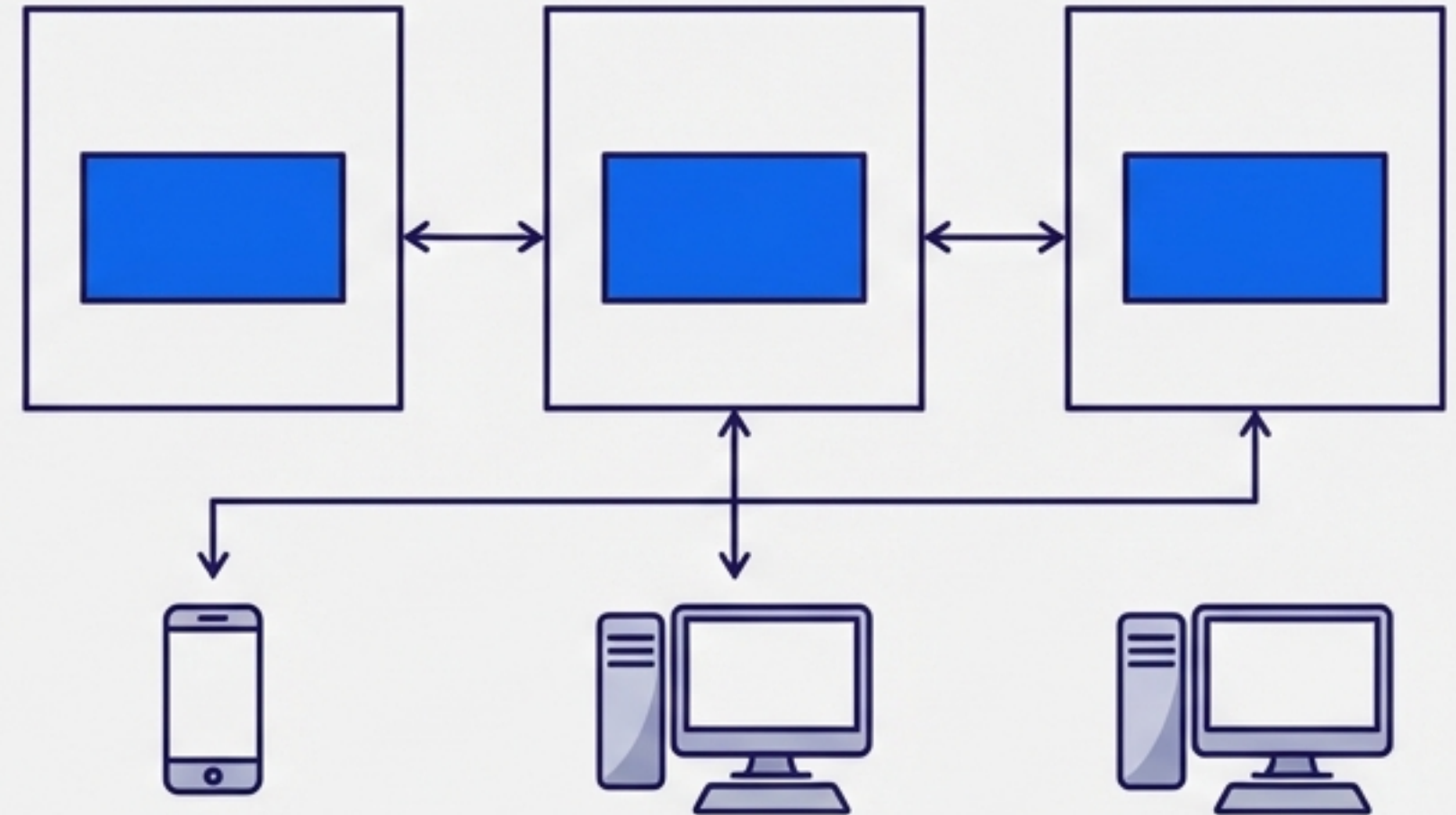
- One or more executing instances of a given type living within a physical container.
- Naming Convention: lowercase.

Deployment Topologies

Centralized



Distributed



Key Takeaway: Modularity frees architectures from monolithic constraints. The exact same logical architecture can be distributed physically to scale independently.

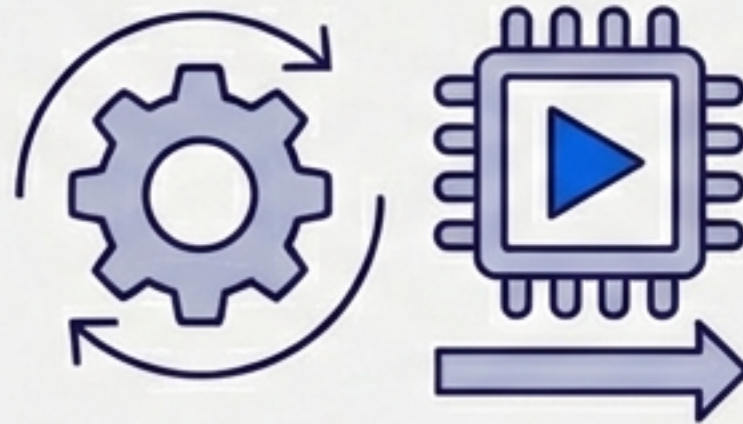
Runtime Execution Roles

User Interface (UI)



Exposes the system to the user. An architecture requires at least one UI component to be accessible. Often layered for different user workflows.

Active Components



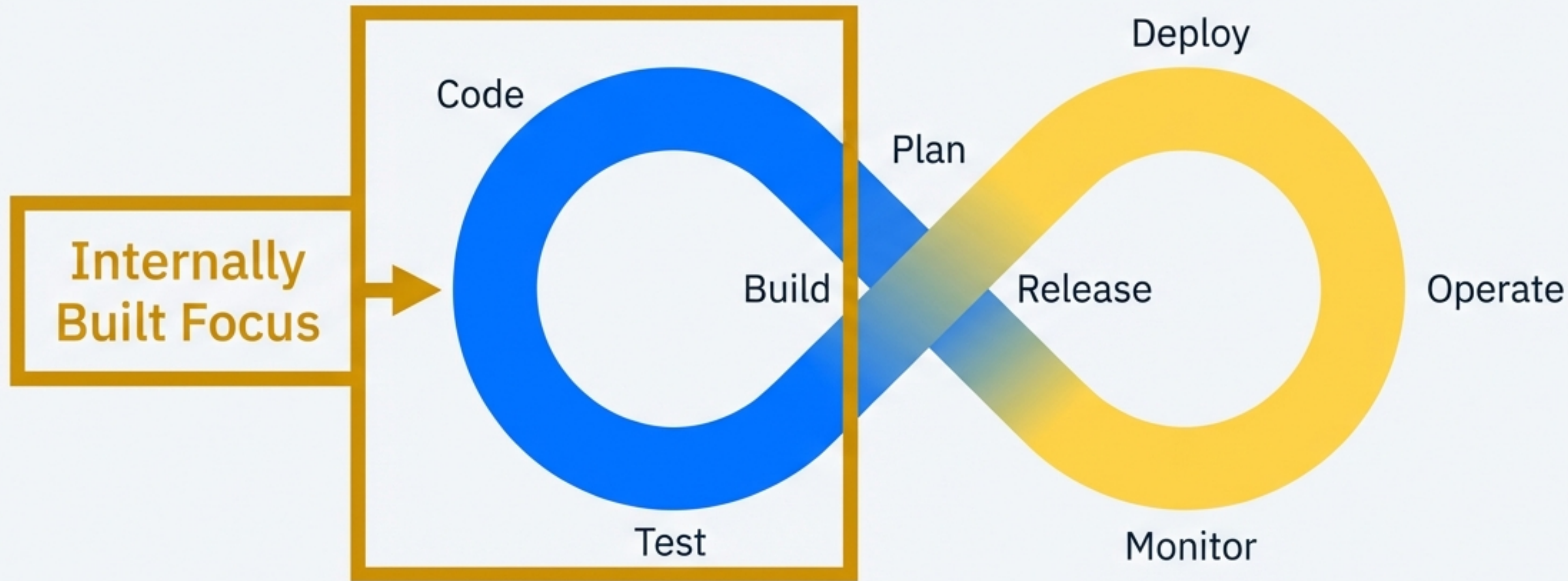
Encapsulate the code that initializes an execution thread. At least one is required to start the application. Multiple active components enable parallelization and multi-core scaling.

Passive Components



Contain logic but do nothing by default. They execute only when their interface is invoked by an active component.

The Lifecycle Branch: Build vs. Buy



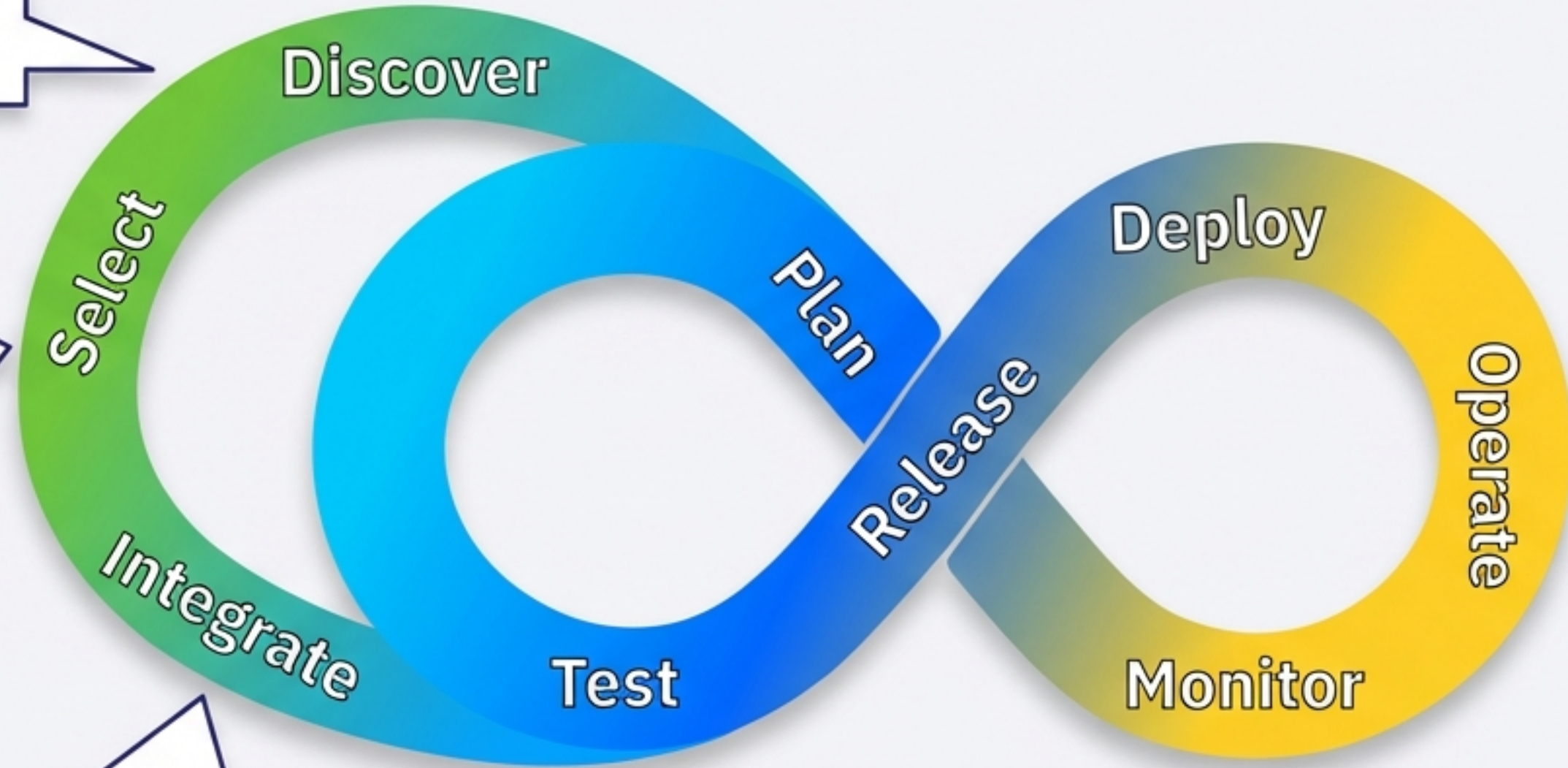
When a new component is modeled, the fundamental architectural decision is whether to write the code in-house (Build) or source it externally (Buy / Open-Source), which profoundly alters the required lifecycle stages.

Sourcing Externally (The 'Buy' Path)

Discover: Finding suitable components via NPM, App Stores, or GitHub.

Select: Evaluating based on price, support, and trust. (Warning: Reputable packages can be compromised via supply chain attacks or malicious handovers).

Integrate: Ensuring the component fits system assumptions (language, OS compatibility, database schema).



Quality Assurance & Release Packaging



Testing & Benchmarking

- Identify system bottlenecks.
- Define workload types and metrics (e.g., response time).
- Benchmark performance against reference targets to detect regressions.

Release

- Turning software into data.
- Packaging compiled code, metadata, and documentation.
- Publishing to an artifact repository for download, installation, or remote invocation.

The Complexity of Memory: Stateful vs. Stateless

Stateless

$$y = f(x)$$

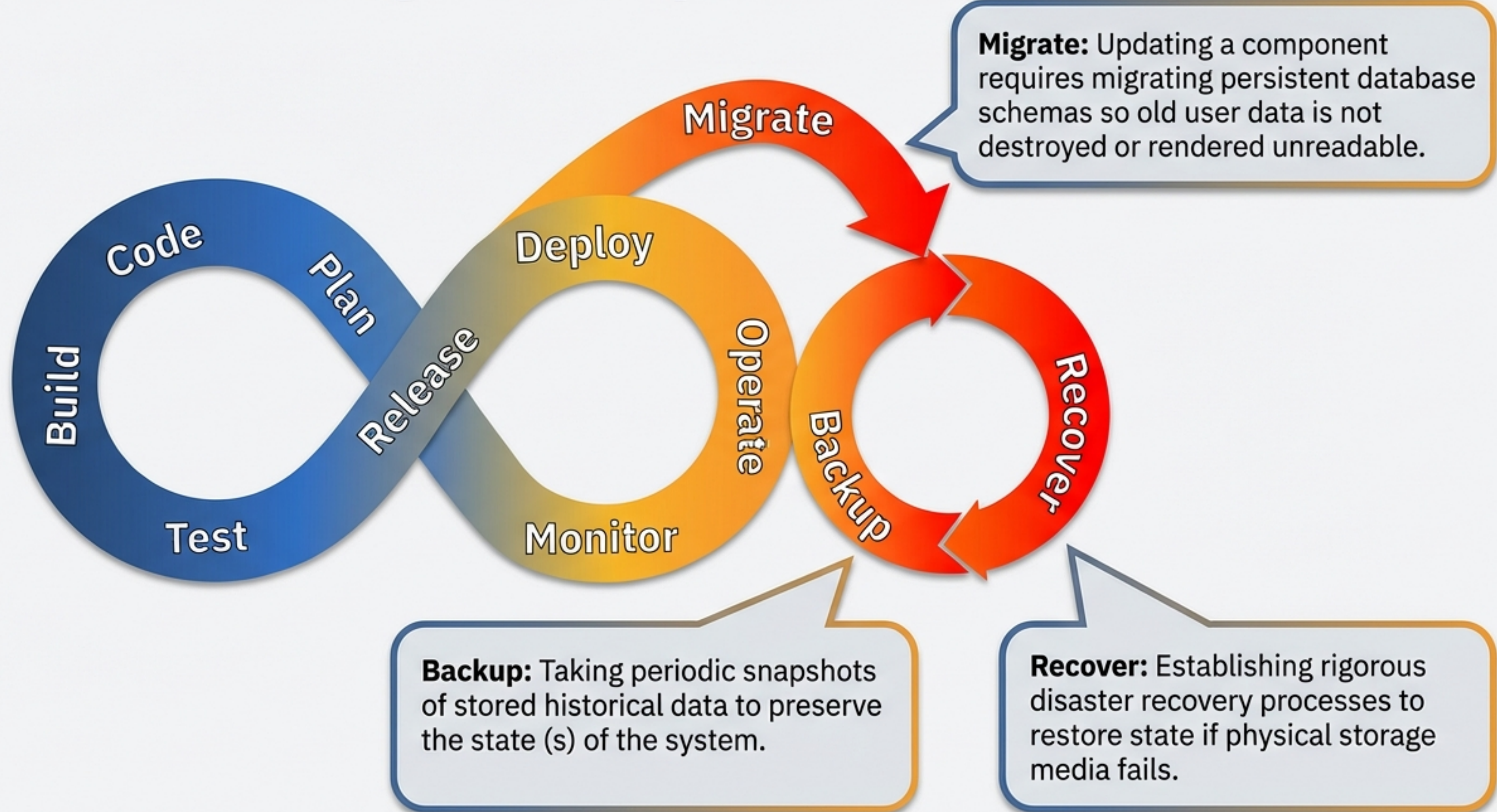
The output (**y**) depends only on the current input (**x**). Highly predictable, memoryless, and fundamentally easy to swap, parallelize, or scale.

Stateful

$$\langle y, s' \rangle = f(x, s)$$

The output (**y**) and the new state (**s'**) depend on both the current input (**x**) and the current historical state (**s**). The component remembers previous inputs, making it far more complex to manage and scale.

The Stateful Lifecycle Complication



The Architectural Checklist

Before introducing a new component into the system architecture, define:

☒ Boundaries: Are the Provided and Required interfaces strictly defined?	☒ Taxonomy: Is it Application-Specific logic or generic Infrastructure?	☒ Sourcing: Will it be Built internally or Sourced/Bought externally? (Is the source trusted?)
☒ Execution: Does it act as a UI, actively spawn threads, or sit passively?	☒ State: Is it Stateless ($y=f(x)$) or Stateful? If Stateful, is there a migration and backup plan?	☒ Topology: Will it be deployed in a Centralized container or Distributed across hosts?